

Secure-FEPRH: Project Ideas & Feature Scope

For Academic Supervisor Review & Approval

Context & Background

A previous graduation project implemented two core features:

- 1. A **code repository** for archiving student projects
- 2. A **text similarity checker** for project proposals (English only)

Secure-FEPRH builds upon and significantly extends this foundation by introducing **Cybersecurity, AI-powered Computer Vision, and Arabic NLP** capabilities — transforming it from a simple archive into a comprehensive, secure academic project lifecycle platform.

What Was Previously Implemented (By Other Students)

#	Feature	Limitations in the Previous Version
1	Basic code/file repository	No encryption, no access control, no file validation
2	Text similarity check (English)	English only — does not support Arabic proposals, which are the majority at the faculty

What We Are Improving (Enhancements to Existing Features)

1. Arabic Language Support for Similarity Detection

- **Previous:** Only English text similarity using basic TF-IDF
- **Our Improvement:** Full Arabic NLP pipeline — including Arabic tokenization, normalization (handling إ/ variants, taa marbuta), Arabic stop-word removal, and Arabic stemming/lemmatization using tools إ/ like Farasa or CAMEL Tools
- **Why it matters:** The vast majority of project proposals and abstracts at the faculty are written in Arabic. Without Arabic support, the similarity engine is effectively useless for most submissions

- **Bonus:** The system will auto-detect whether the submitted text is Arabic or English and apply the correct pipeline accordingly

2. Secure Code Repository (Hardened Version)

- **Previous:** Basic file upload with no protection
- **Our Improvement:**
 - Files encrypted at rest using AES-256 encryption
 - Strict file-type validation — only allowed extensions (`.py` , `.c` , `.cpp` , `.zip` , `.pdf` , etc.)
 - MIME type verification using magic bytes to catch renamed executables
 - Blocking of dangerous file types (`.exe` , `.bat` , `.sh` , `.dll`) to prevent malware uploads
 - SHA-256 hashing for file integrity verification and deduplication
 - File versioning — students can upload multiple versions (v1.0, v1.1, etc.) instead of overwriting

What We Are Adding (Completely New Features)

3. Biometric Face Authentication (Multi-Factor Authentication)

- Users log in with email/password **plus** a face scan via their device camera
- The system extracts a mathematical face encoding (a 128-dimensional number vector) using deep learning and compares it against the stored encoding to verify identity
- **Liveness detection** ensures the user is a real person in front of the camera — not a printed photo or a phone screen — by checking for natural eye blinking and head movement
- Face encodings (numbers only) are stored instead of actual photos to protect student privacy even if the database is breached
- **Fallback mechanism:** If the camera fails (poor lighting, glasses, etc.) after 3 attempts, the system sends a one-time password (OTP) to the user's university email as a secure alternative

4. Role-Based Access Control (RBAC)

- Every user is assigned a role: **Student**, **Supervisor**, **Department Head**, or **System Admin**
- Each role has strictly defined permissions — for example:
 - Students can only upload files and view their own projects
 - Supervisors can view and evaluate only their assigned projects
 - The Department Head has read access to all projects and analytics
 - The System Admin manages user accounts and platform settings
- Every API endpoint on the server enforces role checks — not just the UI — preventing privilege escalation attacks

5. Code Plagiarism Detection (Beyond Text)

- The existing text similarity check catches copied abstracts, but **not copied code**
- Students can easily bypass text-based checks by renaming variables in copied code
- **Our solution:** Analyze the structural logic of the code (its Abstract Syntax Tree / AST), not just the text. Two programs that do the same thing with different variable names will be detected as structurally identical
- This works for Python code natively, and can extend to C/C++/Java via the MOSS (Measure of Software Similarity) service

6. Smart Attendance System (Computer Vision)

- During graduation project discussion/defense sessions, the system uses the room's existing webcam to automatically detect and identify all attendees (students and examination committee members)
- Faces are continuously detected in the camera feed and matched against enrolled face encodings
- A person is marked as "Present" only after being confidently recognized across multiple frames to avoid false detections
- The system generates an **immutable attendance report** — no one can retroactively alter who was present
- Eliminates manual paper-based attendance which is error-prone and easily falsified

7. Built-in Kanban Task Management

- Each approved project automatically gets a task board (similar to Trello) with columns: Backlog, To Do, In Progress, Review, Done
- Supervisors can create tasks and milestones for their students each week
- Students move tasks across columns as they work, giving supervisors visibility into **continuous progress** — not just the final submission
- Supervisors can leave comments and feedback directly on individual tasks
- This encourages an agile, sprint-based workflow that mirrors real industry practices

8. Automated Notification & Alert System

- **Deadline reminders:** Automatic alerts sent 48 hours and 24 hours before submission deadlines
- **Feedback alerts:** Students are notified immediately when their supervisor leaves feedback on a task or uploaded code
- **Security alerts:** If a user's face authentication fails 3+ times, both the user and the admin receive an alert about the suspicious login attempt
- **Session scheduling:** All project members are notified when a discussion session is scheduled
- Notifications are delivered both **in-app** (toast notifications inside the platform) and via **email** to university addresses

9. Analytics Dashboard (Department Head)

- A visual overview panel exclusively for the Department Head, showing:
 - Total number of projects per academic year
 - Distribution of projects by domain (AI, IoT, Web, Embedded, etc.)
 - Most used programming languages across all projects
 - Average similarity scores and trend over time
 - Number of flagged (high-similarity) projects
 - Supervisor workload distribution — how many projects each supervisor is managing
 - Task completion rates across all active projects
- Data is presented through interactive charts and summary cards for quick decision-making

10. Comprehensive Security Hardening

- **Rate limiting** on login endpoints to prevent brute-force password guessing attacks
- **Short-lived JWT tokens** (15-minute expiry) for API session management, with secure refresh token rotation
- **Input sanitization** on all text fields to prevent SQL injection and cross-site scripting (XSS) attacks
- **Account lockout** after 5 consecutive failed login attempts (15-minute cooldown)
- **Full audit logging** — every significant action (login, file upload, file download, proposal approval/rejection) is recorded with timestamp, user ID, and IP address, creating a tamper-evident trail

Summary Table

#	Feature	Status	Category
1	Code/File Repository (Encrypted + Versioned)	 Enhanced	Security + Storage
2	Text Similarity (Arabic + English)	 Enhanced	NLP + AI
3	Biometric Face Authentication (MFA)	 New	Computer Vision + Security
4	Role-Based Access Control (RBAC)	 New	Security
5	Code Plagiarism Detection (AST-based)	 New	AI + Analysis
6	Smart Attendance (Face Recognition)	 New	Computer Vision
7	Kanban Task Management	 New	Project Management

#	Feature	Status	Category
8	Notification & Alert System	 New	Communication
9	Analytics Dashboard	 New	Data Visualization
10	Security Hardening (Rate Limit, JWT, Audit Logs)	 New	Cybersecurity

2 features enhanced from the previous project + **8 completely new features** = **10 total features**

Document Purpose: This document is intended for the academic supervisor to review the proposed feature scope and confirm the team's ability to implement the project. Technical details (stack, algorithms, workflows, diagrams) are available in the full Master Reference document upon request.